

Communicating while you code

Week 5, Lecture 2 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- Why thinking aloud is a scored skill, separate from getting the answer right
- A repeatable edge-case checklist you can run before writing line one
- Three clarifying questions every candidate should ask, and how to ask them well
- Concrete tactics for getting unstuck without going silent

By the end of this lecture you can narrate a 20-minute coding session in interview style, including edge-case enumeration and a recovery move.

Thinking aloud as a skill

What interviewers score that candidates miss

Most candidates prepare for the coding question. The rubric scores four things:

1. **Problem solving:** does the approach work?
2. **Communication:** are you keeping the interviewer informed?
3. **Code quality:** is the code readable and idiomatic?
4. **Verification:** do you test and debug systematically?

Tay (2023): "Try to avoid staying quiet for long periods of time, as it is important to keep the interviewer in your loop and talk them through your approach."

Why silence is expensive

- Silence forces the interviewer to guess whether you are thinking or lost
- A stuck-but-communicating candidate looks like a problem solver
- A stuck-but-silent candidate looks like they gave up
- The interviewer cannot help you if they do not know where you are

Mihailescu (2020): "Most people fail coding interviews not because they cannot code. They fail because they do not have a process. When you get stuck, you need a system for getting unstuck."

Building the narration habit before the interview

The habit does not form under pressure. Build it in practice:

- Solve one problem aloud per day, even when alone
- Use phrases like: "I am thinking about..." and "My concern here is..."
- Record yourself once. You will hear the silences you cannot feel.
- Section this week requires a recorded narrated solve. That recording is practice, not punishment.

Narration is not commentary on code you already know. It is the running account of how you are thinking.

Edge-case enumeration

A checklist that runs before line one

Run this before touching the keyboard. For every problem:

1. **Empty input.** What if the array is empty, the string is empty, $n = 0$?
2. **Single element.** What if there is exactly one element?
3. **Negative numbers.** Are they possible? Does your solution handle them?
4. **Duplicates.** If the input has repeated values, what does that do to your approach?
5. **Overflow.** If you are summing or multiplying, can the intermediate value exceed int range?
6. **Off-by-one.** Is your loop boundary inclusive or exclusive? Does it match the problem?

(Tech Interview Handbook, Tay 2023)

How to state edge cases aloud

Do not just list them. Connect each one to your approach.

"If the input is empty, I will return zero immediately. My loop assumes at least one element, so I want to handle that before entering it."

"The problem says integers but does not specify they are positive. I am going to ask about that."

This shows the interviewer you are thinking about correctness, not just speed to a first pass.

Clarifying questions

Three questions to ask before coding

1. **Input constraints.** "Can the array be empty? Are values always positive? What is the maximum size of n ?"
2. **Output format.** "Should I return the value itself or its index? What should I return if there is no answer?"
3. **Edge behavior.** "If there are multiple valid answers, should I return any one of them or all of them?"

Ask these first, not while your code is half-written. The answers may change your approach entirely.

How to ask without sounding like you are stalling

State what you are doing:

"Before I start coding, I want to confirm a few things about the input."

Then ask one question at a time and wait for the answer. Do not fire all three at once; it sounds rehearsed.

After the answers, say: "OK, with that I am thinking of approaching this with [approach]. Does that seem reasonable to you?"

That last sentence invites feedback before you spend 15 minutes on the wrong path.

What the interviewer is watching

- Candidates who ask zero questions often miss edge cases mid-solution
- Candidates who ask too many questions or use questions to delay coding score poorly on efficiency
- The signal is: one to three focused questions, answered quickly, followed by immediate forward progress

A good clarifying question is one where the answer changes what you write.

Recovery when stuck

The moment most candidates fall apart

You are 15 minutes in. Your approach produces wrong output on a test case. You have no immediate idea why.

What you want to do: freeze, go silent, hope the answer comes.

What scores points: narrate the stuck state and apply a recovery tactic.

Mihailescu (2020): "When you get stuck, you need a system for getting unstuck."

Four recovery tactics, in order

1. **Step back to examples.** Trace through a small concrete input by hand. Write the values on paper or in comments. What does each step actually produce?
2. **Simplify the problem.** Can you solve it if the array has two elements? Three? What breaks when you add the fourth?
3. **Name what you know.** Say aloud: "I know I need to find X. I know the answer is at most Y. I know my current approach fails when Z." Naming constraints often reveals the next step.
4. **Ask for a hint.** Say: "I am stuck on the transition. Is there a direction you can point me in?" This is allowed and often appreciated. Do not pretend to be stuck; ask.

Phrases that keep the interviewer with you

While stuck:

- "I think the issue is in how I handle the boundary condition. Let me trace through with a small example."
- "My current approach is $O(n^2)$. I wonder if I can use a hash map to bring that down."
- "I do not immediately see the transition. Let me try a few small cases and see if a pattern appears."

These phrases show methodology. An interviewer watching these phrases will often give a hint before you even finish the sentence.

Take-aways

1. Thinking aloud is scored separately from correctness. A correct-but-silent solve often scores lower than an incorrect-but-communicating one.
2. Run the edge-case checklist (empty, single, negatives, duplicates, overflow, off-by-one) before writing any code.
3. Ask one to three clarifying questions before coding. Make them focused. After answers, state your approach and invite feedback.
4. When stuck, narrate the stuck state immediately. Then apply a recovery tactic: trace examples, simplify, name constraints, or ask.
5. The habit does not form under pressure. Practice narration aloud every time you solve a problem, starting today.

"Most people fail coding interviews not because they can't code. They fail because they don't have a process. When you get stuck, you need a system for getting unstuck."

Clement Mihailescu, How To Pass Your Coding Interviews (2020)

What's next

- **Section (this week):** Pattern sprint 3, two DP mediums plus a recorded narrated solve with peer feedback
- **Reading:** Week 5 reading covers the DP example end to end and the communication checklist
- **HW2 (30-problem LeetCode sprint):** due this week
- **HW3 (recorded mock coding interview):** now out, due Week 7
- **Next week:** System design as a conversation (Week 6)